Progress Report – Phase 1

Tower Defense Game Project

Authors:
Adham AHMED SALAH ALI
William Kennedy NDE FOTSO

Supervisor: Yohan Fougerolle
Department of Robotics and Instrumentation
Polytech Dijon — Université de Bourgogne

Academic Year: 2025

Project Repository:
`github.com/AdhamElkomi/Tower-Defense-Game`

Contents

1. Introduction

As part of our semester 7 capstone project, we are developing a **Tower Defense game** in C++ using the **SFML 3** library. This report presents the progress achieved so far, focusing on the system architecture, the main menu, random map generation, and early gameplay visuals.

2. Project Structure

2.1. Software Architecture

The software architecture of the *Tower Defense Game* has been carefully structured to ensure modularity, scalability, and clarity. The project follows a layered design where each module has a well-defined responsibility. This section provides an overview of the directory structure and explains the roles of the main components.

2.1.1 Project Structure

```
.
CMakeLists.txt                # Build configuration
README.md / LICENSE
assets/                       # Runtime resources
  fonts/                     # Typography (Roboto)
  images/                    # Static backgrounds, UI
  sounds/                    # SFX and music
  tiles/                     # Tile atlas (terrain_atlas.png)
  trees/                     # Tree sprites (tree1.png..treeN.png)
docs/                         # Reports, UML diagrams, captures
include/                      # Public headers
  App.hpp, GameScene.hpp, Menu.hpp, ...
src/                          # Implementations (.cpp)
  App.cpp, GameScene.cpp, Menu.cpp, ...
tests/                        # Unit/regression tests
vcpkg/                        # Dependencies (SFML 3)
.github/, .gitignore          # CI/CD and versioning
```

Figure 2.1: Project structure of the Tower Defense Game.

This tree illustrates the clear separation between assets, source code, headers, build configuration, and documentation. The `include/` and `src/` directories mirror each other to maintain a clean interface/implementation pattern.

2.1.2 Main Components

- **App**: Entry point and orchestrator. Owns the main window, manages the game loop, and transitions between scenes.

- **Scene (base)**: Abstract interface with `handleInput`, `update`, and `draw`. Ensures consistency across game states.
- **MenuScene**: Main menu with background, panel, start/settings/exit buttons, and sliders.
- **GameScene**: Core gameplay layer. Loads maps, generates terrain, and manages decorative systems such as trees.
- **UI Widgets**:
  - `Button`, `CircleButton`: Reusable clickable widgets.
  - `Slider`: Used for adjusting continuous parameters (e.g., audio volume).
- **Map & Systems**:
  - `Map`: Grid representation of the game world.
  - `MapGenerator`: Procedural map generator ensuring consistent entry/exit paths.
  - `TileMap`: Efficient renderer of the grid using a tile atlas.
  - `TreeSystem`: Places tree sprites at random positions, avoiding collision with paths.
- **AudioManager**: Loads, manages, and plays background music and sound effects with volume control.

2.1.3 Design Principles

- **Modularity**: Each component is isolated and tested independently.
- **Scalability**: Adding new scenes (e.g., Pause, Game Over) or new systems (e.g., enemy spawners) requires minimal changes.
- **Maintainability**: Strict mirroring between headers (`include/`) and source files (`src/`) ensures readability.
- **Performance**: `TileMap` leverages batched rendering (`sf::VertexArray`) to minimize draw calls.

2.2. UML Diagram

Figure 2.2 illustrates the UML diagram of the project.

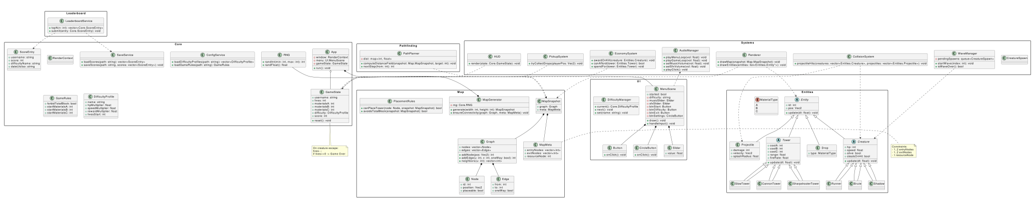

Figure 2.2: UML structure of the Tower Defense project

3. Functional Progress

3.1. Main Menu

The main menu has been implemented with a professional design:

- **Start, Difficulty, Exit buttons**, each with distinct backgrounds and shadow effects.
- **Circular Settings button** opening sliders for both music and sound effects volume.
- **Background music** loops continuously and switches when starting the game.

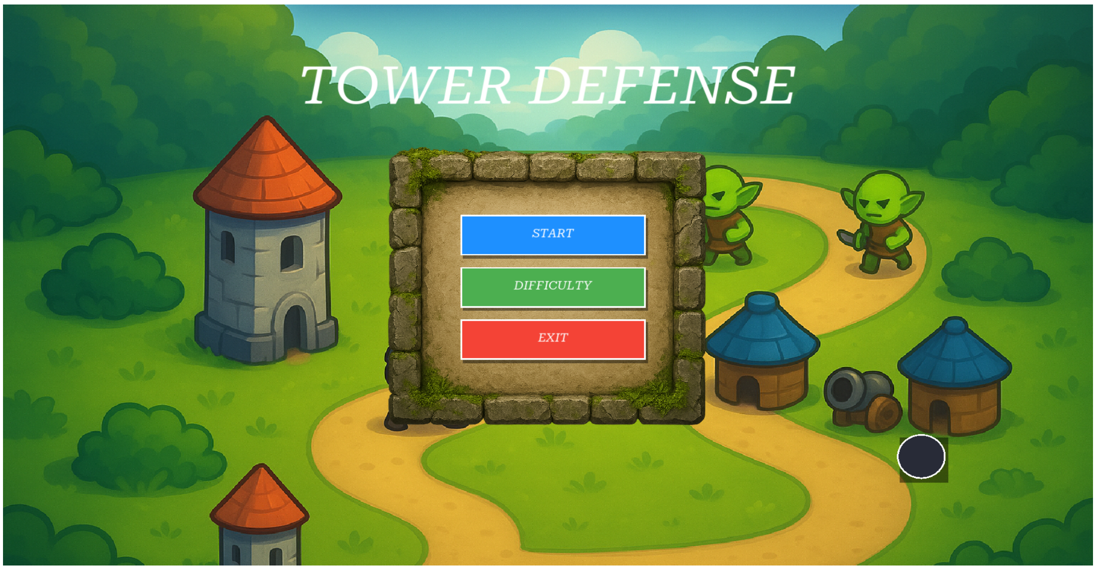

Figure 3.1: Main menu interface

3.2. Procedural Map Generation

We integrated a procedural map generator with the following features:

- **1–2 entries and 1–2 exits**, placed randomly at each launch.
- Roads are generated to ensure valid connectivity.
- **Random decorative elements** (trees, bushes, rocks) placed outside roads.

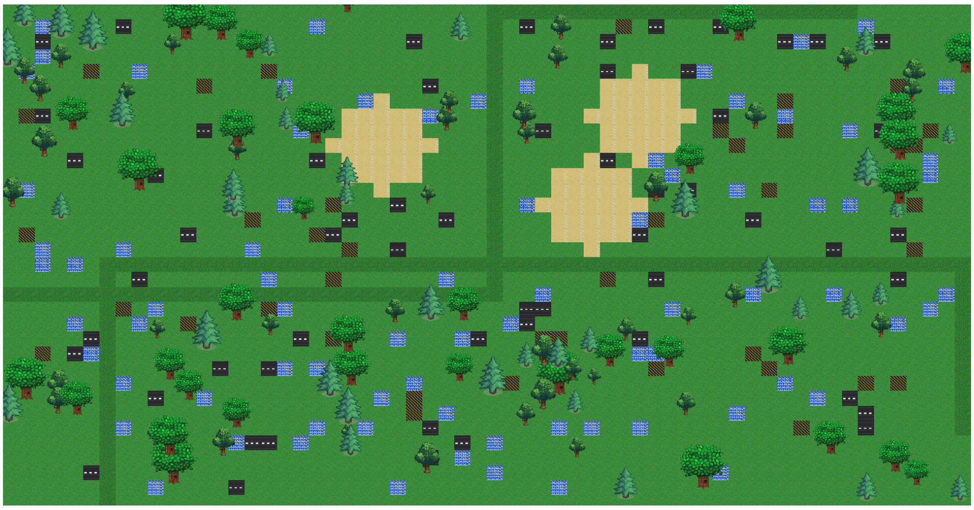

Figure 3.2: Example of a procedurally generated map

3.3. Trees and Decorative System

We split our tree sprites (`tree1.png`, `tree2.png`, `tree3.png`) and developed a random tree placement system to enhance natural variety.

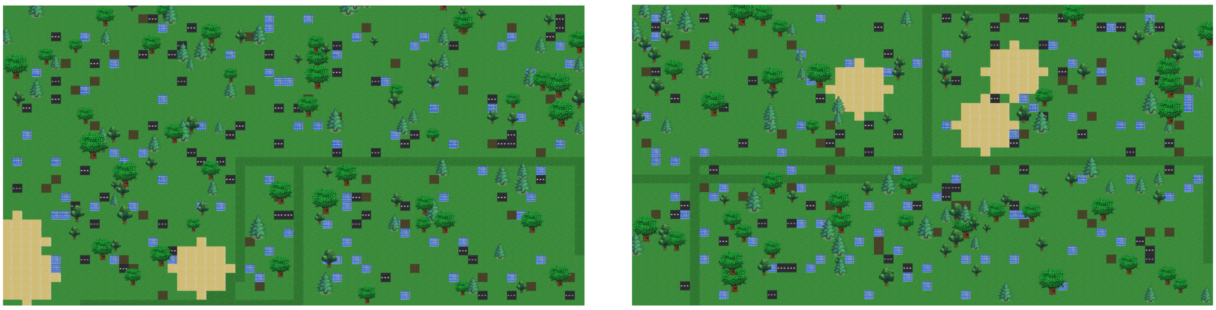

Figure 3.3: Examples of randomly generated maps with varied tree placements.

4. Conclusion and Next Steps

At this stage, we have achieved:

- A **robust modular architecture**.
- A **fully functional and styled menu**.
- A **procedural map generator**.
- A working **decorative system with trees**.

The next steps will focus on:

- Implementing the **player's life system** and **scoring**.
- Managing enemy waves and defensive towers.
- Saving high scores with usernames.
- Enhancing visuals with improved textures and animations.

Bibliography

[1] SFML Development Team, *Simple and fast multimedia library (sfml) 3 — official documentation*, 2024. [Online]. Available: https://www.sfml-dev.org/tutorials/3.0/getting-started/linux.

[2] Kitware, *Cmake — build system documentation*, 2024. [Online]. Available: https://cmake.org/.

[3] Google Fonts, *Roboto typeface family*, 2011. [Online]. Available: https://fonts.google.com/specimen/Roboto.

[4] OpenGameArt Community, *Free 2d rpg tileset (grass, path, water, rocks)*, https://opengameart.org/, Used for terrain tiles in the Tower Defense Game, 2024.

[5] OpenGameArt Community, *Royalty-free game music collection*, https://opengameart.org/, Used for free background tracks and ambient music, 2024.

[6] OpenGameArt Community, *Pixel art trees (tree1, tree2, tree3)*, https://www.google.com, Used for decorative tree sprites, 2024.

[7] OpenAI ChatGPT, *Technical and writing assistance for the tower defense project*, https://chat.openai.com/, Consulted for guidance and clarification during development, 2025.